



Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра суперкомпьютеров и квантовой информатики

Параллельная обработка больших графов
Параллельная реализация алгоритма Брандеса решения
задачи расчета betweenness centrality

Группа: 523

Повжик Юрий Максимович

Вариант: MPI-1, point-to-point communication

Москва, 2026

Содержание

1. Характеристики вычислительной системы	3
2. Описание решения.....	4
2.1 Программный код	4
2.2 Описание программной реализации.....	8
2.3 Проверка корректности программы.....	10
3. Параметры запуска программы	11
4. Результаты измерений производительности.....	12
5. Анализ полученных результатов.....	13
6. Применение алгоритма Reverse Cuthill McKee	14
6.1 Программный код	14
6.2 Описание программной реализации.....	18
6.3 Проверка корректности программы с оптимизацией	19
6.4 Результаты измерения производительности.....	20
6.5 Сравнение производительности	21

1. Характеристики вычислительной системы

Запуск задания производился на Polus со следующими характеристиками:

Пиковая производительность	55.84 TFlop/s
Производительность (Linpack)	40.39 TFlop/s
Вычислительных узлов	5

На каждом узле:

Процессоры IBM Power 8	2
NVIDIA Tesla P100	2
Число процессорных ядер	20
Число потоков на ядро	8
Оперативная память	256 Гбайт

Коммуникационная сеть	Infiniband / 100 Gb
Система хранения данных	GPFS
Операционная система	Linux Red Hat 7.5

Память:

На одном узле — 2 сокета

На каждом сокете:

4 планок DDR-2400 по 32 GB (Всего 128 GB)

2 кентавра (L4 кэш + контроллер памяти)

Кентавр - 4 канала памяти

(8 каналов на сокет, 16 каналов на узел)

L1 кэш – 64KB

L2 кэш – 512KB

L3 кэш – 8 MB/ядро

L4 кэш – 16 MB на кентавр (32 MB на сокет)

Пиковая пропускная способность на сокете – 153.6 GB/s

Пиковая пропускная способность на узле – 307.2 GB/s

2. Описание решения

2.1 Программный код

```
/*Структуры для обмена данными*/
struct bfs_pkg { vertex_id_t v_glob; double sigma; };
struct bp_pkg { vertex_id_t v_glob; double sigma; double delta; };

void brandes_distributed(graph_t* G, double* local_results) {
    /*инициализация*/
    int rank = G->rank, size = G->nproc;
    vertex_id_t N = G->n;
    vertex_id_t local_n = G->local_n;

    for (vertex_id_t i = 0; i < local_n; i++)
        local_results[i] = 0.0;

    vector<int> local_d(local_n);
    vector<double> local_sigma(local_n);
    vector<double> local_delta(local_n);
    vector<vector<bfs_pkg>> send_bufs(size);

    MPI_Datatype MPI_BFS_PKG, MPI_BP_PKG;
    MPI_Type_contiguous(sizeof(bfs_pkg), MPI_BYTE, &MPI_BFS_PKG);
    MPI_Type_commit(&MPI_BFS_PKG);
    MPI_Type_contiguous(sizeof(bp_pkg), MPI_BYTE, &MPI_BP_PKG);
    MPI_Type_commit(&MPI_BP_PKG);

    /*обход из каждой вершины графа*/
    for (vertex_id_t s = 0; s < N; s++) {
        fill(local_d.begin(), local_d.end(), -1);
        fill(local_sigma.begin(), local_sigma.end(), 0.0);
        fill(local_delta.begin(), local_delta.end(), 0.0);

        bool made_progress = false;
        if (VERTEX_OWNER(s, N, size) == rank) {
            vertex_id_t s_loc = VERTEX_LOCAL(s, N, size, rank);
            local_d[s_loc] = 0;
            local_sigma[s_loc] = 1.0;
            made_progress = true;
        }

        int cur_d = 0;
        bool global_active = true;
        MPI_Allreduce(&made_progress, &global_active, 1, MPI_C_BOOL, MPI_LOR, MPI_COMM_WORLD);

        /*BFS*/
        while (global_active) {
            for (int i = 0; i < size; i++)
```

```

        send_bufs[i].clear();
made_progress = false;

for (vertex_id_t u_loc = 0; u_loc < local_n; u_loc++) {
    if (local_d[u_loc] == cur_d) {
        double u_sigma = local_sigma[u_loc];
        for (edge_id_t e = G->rowsIndices[u_loc]; e < G->rowsIndices[u_loc + 1]; e++) {
            vertex_id_t v_glob = G->endV[e];
            int v_owner = VERTEX_OWNER(v_glob, N, size);
            if (v_owner == rank) {
                vertex_id_t v_loc = VERTEX_LOCAL(v_glob, N, size, rank);
                if (local_d[v_loc] == -1) {
                    local_d[v_loc] = cur_d + 1;
                    local_sigma[v_loc] = u_sigma;
                    made_progress = true;
                } else if (local_d[v_loc] == cur_d + 1) local_sigma[v_loc] += u_sigma;
            } else {
                send_bufs[v_owner].push_back({v_glob, u_sigma});
            }
        }
    }
}

vector<int> s_counts(size), r_counts(size);
for (int i = 0; i < size; i++)
    s_counts[i] = send_bufs[i].size();

MPI_Alltoall(s_counts.data(), 1, MPI_INT, r_counts.data(), 1, MPI_INT, MPI_COMM_WORLD);

vector<bfs_pkg> r_data;
int total_recv = 0;
for (int count: r_counts)
    total_recv += count;
r_data.resize(total_recv);

vector<MPI_Request> requests;
vector<int> r_dis(size, 0);
for (int i = 1; i < size; i++)
    r_dis[i] = r_dis[i-1] + r_counts[i-1];

for (int i = 0; i < size; i++) {
    if (r_counts[i] > 0) {
        MPI_Request req;
        MPI_Irecv(&r_data[r_dis[i]], r_counts[i], MPI_BFS_PKG, i, 0, MPI_COMM_WORLD, &req);
        requests.push_back(req);
    }
}

for (int i = 0; i < size; i++) {
    if (s_counts[i] > 0) {

```

```

        MPI_Request req;
        MPI_Isend(send_bufs[i].data(), s_counts[i], MPI_BFS_PKG, i, 0, MPI_COMM_WORLD, &req);
        requests.push_back(req);
    }
}
MPI_Waitall(requests.size(), requests.data(), MPI_STATUSES_IGNORE);

for (const auto& pkg: r_data) {
    vertex_id_t v_loc = VERTEX_LOCAL(pkg.v_glob, N, size, rank);
    if (local_d[v_loc] == -1) {
        local_d[v_loc] = cur_d + 1;
        local_sigma[v_loc] = pkg.sigma;
        made_progress = true;
    } else if (local_d[v_loc] == cur_d + 1) local_sigma[v_loc] += pkg.sigma;
}

MPI_Allreduce(&made_progress, &global_active, 1, MPI_C_BOOL, MPI_LOR, MPI_COMM_WORLD);
cur_d++;
}

/*Обратный ход*/
for (int lv = cur_d - 1; lv > 0; lv--) {
    vector<bp_pkg> level_data;
    for(vertex_id_t i=0; i<local_n; i++) {
        if(local_d[i] == lv) {
            level_data.push_back({VERTEX_TO_GLOBAL(i, N, size, rank), local_sigma[i],
local_delta[i]});
            local_results[i] += local_delta[i];
        }
    }

    int local_count = level_data.size();
    vector<int> r_counts(size);
    MPI_Allgather(&local_count, 1, MPI_INT, r_counts.data(), 1, MPI_INT, MPI_COMM_WORLD);

    int total_lv = 0;
    vector<int> r_dis(size, 0);
    for (int i = 0; i < size; i++) {
        r_dis[i] = total_lv;
        total_lv += r_counts[i];
    }

    vector<bp_pkg> recv_level(total_lv);
    vector<MPI_Request> requests;

    for (int i = 0; i < size; i++) {
        if (r_counts[i] > 0) {
            MPI_Request req;
            MPI_Irecv(&recv_level[r_dis[i]], r_counts[i], MPI_BP_PKG, i, 1, MPI_COMM_WORLD, &req);
            requests.push_back(req);

```

```

    }
}
for (int i = 0; i < size; i++) {
    if (local_count > 0) {
        MPI_Request req;
        MPI_Isend(level_data.data(), local_count, MPI_BP_PKG, i, 1, MPI_COMM_WORLD, &req);
        requests.push_back(req);
    }
}
MPI_Waitall(requests.size(), requests.data(), MPI_STATUSES_IGNORE);

sort(recv_level.begin(), recv_level.end(), [](const bp_pkg& a, const bp_pkg& b) {
    return a.v_glob < b.v_glob;
});

for (vertex_id_t u_loc = 0; u_loc < local_n; u_loc++) {
    if (local_d[u_loc] == lv - 1) {
        double u_sigma = local_sigma[u_loc], d_sum = 0;
        for (edge_id_t e = G->rowsIndices[u_loc]; e < G->rowsIndices[u_loc + 1]; e++) {
            vertex_id_t v_glob = G->endV[e];
            auto it = lower_bound(recv_level.begin(), recv_level.end(), v_glob,
                                [](const bp_pkg& p, vertex_id_t val){ return p.v_glob < val;
                                });
            if (it != recv_level.end() && it->v_glob == v_glob)
                d_sum += (u_sigma / it->sigma) * (1.0 + it->delta);
        }
        local_delta[u_loc] += d_sum;
    }
}
}

/*Нормализация, так как граф неориентированный*/
for (vertex_id_t i = 0; i < local_n; i++)
    local_results[i] /= 2.0;

MPI_Type_free(&MPI_BFS_PKG); MPI_Type_free(&MPI_BP_PKG);
}

```

2.2 Описание программной реализации

В данной работе реализована распределенная версия алгоритма Брандеса для вычисления посреднической центральности (Betweenness Centrality) вершин графа с использованием библиотеки MPI. Реализация ориентирована на работу в системах с распределенной памятью, где граф разделен между вычислительными узлами.

Распределение данных

Граф распределяется между процессами по вершинам. Каждый процесс несет ответственность за подмножество из N/p вершин и хранит соответствующий им фрагмент списка смежности. Для каждой локальной вершины процесс поддерживает следующие массивы:

- `local_d` — кратчайшие расстояния от текущего источника s
- `local_sigma` — количество кратчайших путей, проходящих через вершину
- `local_delta` — коэффициенты зависимости для обратного прохода
- `local_results` — итоговые накопленные значения центральности для локальных вершин

Общая схема алгоритма

Алгоритм выполняется для каждой вершины графа s как источника и состоит из двух фаз:

1. Прямой обход
2. Обратное распространение зависимостей

Прямой обход

Для каждой вершины-источника:

- инициализируются массивы расстояний (-1), числа путей (0) и зависимостей (0)
- для источника устанавливается расстояние = 0, $\sigma = 1$

Далее выполняется уровневый обход в ширину (BFS):

- на каждом шаге рассматриваются вершины текущего уровня `cur_d`
- для каждой вершины:
 - локальные соседи обрабатываются напрямую
 - удалённые помещаются в буферы отправки (`send_bufs`)

Обмен сообщениями:

- обмен размерами сообщений через `MPI_Alltoall`
- обмен через `MPI_Isend`, `MPI_Irecv`
- данные передаются в структуре:

```
bfs_pkg { vertex_id_t v_glob; double sigma; };
```


После получения:

- обновляются расстояния и значения σ
- если вершина достигнута впервые — фиксируется уровень
- если найден альтернативный кратчайший путь — увеличивается σ

Итерации BFS продолжаются до тех пор, пока хотя бы один процесс в системе имеет активные вершины на текущем уровне. Проверка осуществляется через MPI_Allreduce с операцией MPI_LOR

Обратное распространение зависимостей

Обработка выполняется по уровням в обратном порядке:

1. **Формирование пакетов уровня:** Процесс собирает данные всех локальных вершин текущего уровня lv в структуру `bp_pkg { vertex_id_t v_glob; double sigma; double delta; }`;
2. **Глобальный обмен уровнями:** Все процессы обмениваются данными о своих вершинах уровня lv . Для этого используется MPI_Allgather для получения размеров и неблокирующие MPI_Isend/MPI_Irecv для передачи самих структур. В результате каждый процесс получает полную информацию о текущем слое графа.
3. **Локальный расчет:** Полученные данные сортируются по глобальному индексу для обеспечения возможности бинарного поиска.
4. **Вычисление зависимости:** Для каждой локальной вершины на уровне $lv - 1$ рассматриваются её соседи. Если сосед присутствует в полученном списке уровня lv (является потомком в дереве кратчайших путей), вычисляется его вклад в значение δ данной вершины по формуле Брандеса.
5. **Накопление результата:** Вычисленные значения δ суммируются и добавляются в итоговый массив `local_results`.

Особенности реализации

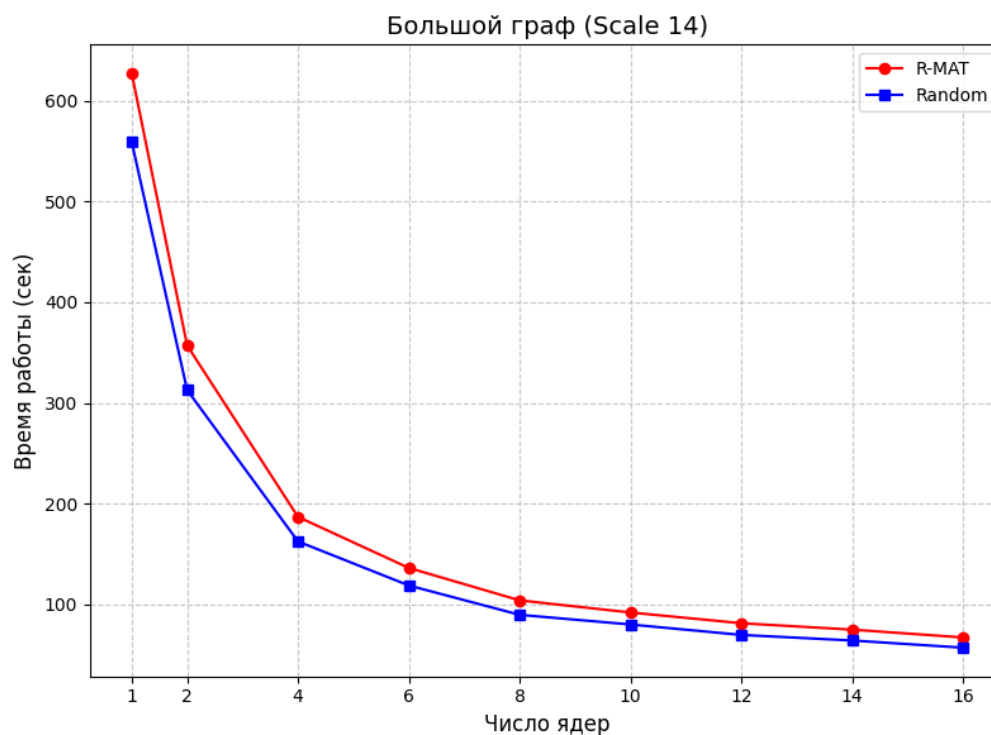
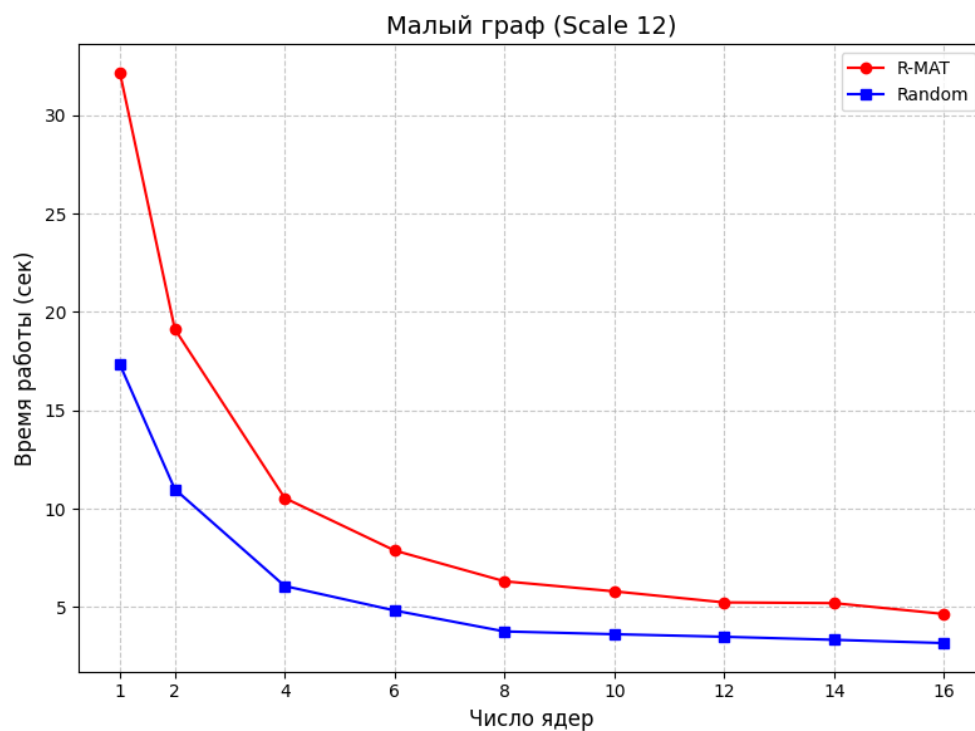
- Используются неблокирующие коммуникации типа точка точка: MPI_Isend, MPI_Irecv
- Синхронизация выполняется через MPI_Allreduce, MPI_Allgather

2.3 Проверка корректности программы

```
via@LAPTOP-5613KMS7:/mnt/c/Programs/C/VS_Code/2026-PLGP-BC/2023-PLGP-BC$ make
mpicxx -Wall -O3 -o solution_mpi.o -c solution_mpi.cpp
In file included from solution_mpi.cpp:1:
defs.h:24: warning: "UINT32_MAX" redefined
   24 | #define UINT32_MAX 0xFFFFFFFF
      |
In file included from /usr/lib/gcc/x86_64-linux-gnu/11/include/stdint.h:9,
      from defs.h:6,
      from solution_mpi.cpp:1:
/usr/include/stdint.h:129: note: this is the location of the previous definition
   129 | # define UINT32_MAX          (4294967295U)
      |
mpicxx main_mpi.o solution_mpi.o graph_tools.o gen_RMAT_mpi.o gen_random_mpi.o -o solution_mpi -O3
via@LAPTOP-5613KMS7:/mnt/c/Programs/C/VS_Code/2026-PLGP-BC/2023-PLGP-BC$ mpirun -n 4 ./solution_mpi -in rmat-12
start algorithm...
iter number = 0 finished. Time is 10.11398 sec.
algorithm iterations finished.
rmat-12 vertices = 4096, edges = 131072, nIters = 1, time: min = 10.11398, avg = 10.11398, max = 10.11398
Time = 10.11398 sec.
via@LAPTOP-5613KMS7:/mnt/c/Programs/C/VS_Code/2026-PLGP-BC/2023-PLGP-BC$ ./gen_valid_info -in rmat-12
via@LAPTOP-5613KMS7:/mnt/c/Programs/C/VS_Code/2026-PLGP-BC/2023-PLGP-BC$ ./validation -ans rmat-12.ans -res rmat-12.res
Accepted
```


4. Результаты измерений производительности

При масштабировании от 2 до 16 ядер использовались два вычислительных узла, на каждом из которых задействовалось одинаковое количество ядер: от 1 до 8 соответственно. Запуск на 1 ядре выполнялся на одном узле.



5. Анализ полученных результатов

На основании проведенных экспериментов и анализа полученных временных показателей можно сделать следующие выводы:

Эффективность распараллеливания

Реализованный алгоритм демонстрирует устойчивое снижение времени выполнения при увеличении числа ядер. Наибольший прирост производительности, близкий к линейному, наблюдается в диапазоне от 2 до 8 ядер.

При увеличении размера графа наблюдается рост эффективности. Это объясняется тем, что с ростом объема вычислений доля накладных расходов MPI-коммуникаций снижается.

Влияние топологии графа

Случайные графы (Random): показали наилучшее время обработки. Равномерное распределение степеней вершин обеспечивает сбалансированную вычислительную нагрузку между процессами.

Графы R-MAT: время обработки остается стабильно выше, чем у случайных графов. Однако при увеличении размера графа разрыв во времени выполнения между R-MAT и Random сократился с 1.5–1.7х на малых данных до 1.15–1.2х на больших. Тем не менее дисбаланс нагрузки, вызванный наличием вершин-хабов, остается главным сдерживающим фактором для R-MAT графов.

6. Применение алгоритма Reverse Cuthill McKee

В качестве оптимизации была выбрана предобработка графа алгоритмом Reverse Cuthill-McKee (RCM), целью которой является минимизация ширины полосы (bandwidth) матрицы смежности графа. Это позволяет значительно уменьшить количество кэш-промахов за счет концентрации ненулевых элементов вблизи главной диагонали матрицы, что обеспечивает высокую пространственную локальность данных. В результате обращения процессора к массивам в процессе выполнения алгоритма Брандеса становятся более последовательными, что минимизирует простои вычислительных ядер при ожидании данных из основной памяти.

6.1 Программный код

В main.cpp добавили apply_local_rcm и исправили порядок записи результата в файл

```
vertex_id_t get_any_local_n(vertex_id_t TotVertices, int size, int rank) {
    vertex_id_t mod_size = TotVertices % size;
    vertex_id_t div_size = TotVertices / size;
    return (rank < mod_size) ? (div_size + 1) : div_size;
}

vertex_id_t* apply_local_rcm(graph_t* G) {
    vertex_id_t ln = G->local_n;

    struct NodeDeg { vertex_id_t id; vertex_id_t deg; };
    vector<NodeDeg> sorted_nodes(ln);
    vector<vertex_id_t> degrees(ln);

    for (vertex_id_t i = 0; i < ln; i++) {
        vertex_id_t deg = G->rowsIndices[i+1] - G->rowsIndices[i];
        degrees[i] = deg;
        sorted_nodes[i] = {i, deg};
    }

    sort(sorted_nodes.begin(), sorted_nodes.end(), [](const NodeDeg& a, const NodeDeg& b) {
        return a.deg < b.deg;
    });

    vector<vertex_id_t> new_order;
    new_order.reserve(ln);
    vector<bool> visited(ln, false);
    vertex_id_t search_ptr = 0;

    while (true) {
        int s = -1;
        while (search_ptr < ln) {
            if (!visited[sorted_nodes[search_ptr].id]) {
                s = sorted_nodes[search_ptr].id;
            }
            search_ptr++;
        }
        new_order.push_back(s);
        visited[s] = true;
    }
}
```

```

        break;
    }
    search_ptr++;
}
if (s == -1) break;

vector<vertex_id_t> current_level;
current_level.push_back(s);
visited[s] = true;
new_order.push_back(s);

while (!current_level.empty()) {
    sort(current_level.begin(), current_level.end(), [&](vertex_id_t a, vertex_id_t b) {
        return degrees[a] < degrees[b];
    });

    vector<vertex_id_t> next_level;
    for (vertex_id_t u : current_level) {
        vector<pair<vertex_id_t, vertex_id_t>> neighbors;
        for (edge_id_t e = G->rowsIndices[u]; e < G->rowsIndices[u+1]; e++) {
            vertex_id_t v_glob = G->endV[e];
            if (VERTEX_OWNER(v_glob, G->n, G->nproc) == G->rank) {
                vertex_id_t v_loc = VERTEX_LOCAL(v_glob, G->n, G->nproc, G->rank);
                if (!visited[v_loc]) {
                    visited[v_loc] = true;
                    neighbors.push_back({degrees[v_loc], v_loc});
                }
            }
        }

        sort(neighbors.begin(), neighbors.end());
        for (auto& p : neighbors) {
            next_level.push_back(p.second);
            new_order.push_back(p.second);
        }
    }
    current_level = next_level;
}

reverse(new_order.begin(), new_order.end());

vector<vertex_id_t> new_to_old_id(ln);
for(vertex_id_t i=0; i<ln; i++) {
    new_to_old_id[i] = VERTEX_TO_GLOBAL(new_order[i], G->n, G->nproc, G->rank);
}

vector<vertex_id_t> all_new_to_old_id(G->n);
vector<int> recv_counts(G->nproc), displs(G->nproc);
for(int i=0; i<G->nproc; i++) {

```

```

        recv_counts[i] = get_any_local_n(G->n, G->nproc, i);
        displs[i] = (i == 0) ? 0 : displs[i-1] + recv_counts[i-1];
    }
    MPI_Allgatherv(new_to_old_id.data(), ln, MPI_UNSIGNED, all_new_to_old_id.data(), recv_counts.data(),
displs.data(), MPI_UNSIGNED, MPI_COMM_WORLD);

    vector<vertex_id_t> old_to_new_id(G->n);
    for(vertex_id_t i=0; i<G->n; i++) {
        old_to_new_id[all_new_to_old_id[i]] = i;
    }

    for (edge_id_t e = 0; e < G->local_m; e++) {
        G->endV[e] = old_to_new_id[G->endV[e]];
    }

    edge_id_t* n_rowsIndices = new edge_id_t[ln + 1];
    vertex_id_t* n_endV = new vertex_id_t[G->local_m];
    vertex_id_t* mapping_to_return = new vertex_id_t[ln];

    edge_id_t cur_e = 0; n_rowsIndices[0] = 0;
    for (vertex_id_t i = 0; i < ln; i++) {
        vertex_id_t old_u_idx = new_order[i];
        mapping_to_return[old_u_idx] = i;
        for (edge_id_t e = G->rowsIndices[old_u_idx]; e < G->rowsIndices[old_u_idx+1]; e++) {
            n_endV[cur_e++] = G->endV[e];
        }
        n_rowsIndices[i+1] = cur_e;
    }

    delete[] G->rowsIndices;
    delete[] G->endV;
    G->rowsIndices = n_rowsIndices;
    G->endV = n_endV;

    for (vertex_id_t i = 0; i < ln; i++) {
        sort(G->endV + G->rowsIndices[i], G->endV + G->rowsIndices[i+1]);
    }

    return mapping_to_return;
}

int main(int argc, char **argv)
{
    graph_t g;
    double start_ts, finish_ts;
    double *alg_time;
    /* initializing */
    init(argc, argv, &g);
    vertex_id_t* old_to_new = apply_local_rcm(&g);

```



```

...

print0(g.rank, "algorithm iterations finished.");
double *ordered_result = new double[g.local_n];
for (vertex_id_t i = 0; i < g.local_n; i++) {
    ordered_result[i] = result[old_to_new[i]];
}
write_output_information(ordered_result, g.local_n, outFilename);

delete[] old_to_new;
delete[] ordered_result;

...
}

```

6.2 Описание программной реализации

В начале процедуры на основе исходного массива смещений CRS ($G \rightarrow rowsIndices$) вычисляются степени всех локальных вершин. На их основе формируется вспомогательный массив пар «индекс-степень», который сортируется по возрастанию степеней. Это позволяет в дальнейшем находить стартовые узлы для новых компонент связности за константное время с помощью указателя `search_ptr`.

Основная логика переупорядочивания реализована через алгоритм RCM, формирующий вектор перестановок `new_order`:

- для каждой компоненты связности локального подграфа из предварительно отсортированного списка выбирается не посещенная ранее вершина с минимальной степенью, что обеспечивает начало обхода с «периферии».
- вершины обрабатываются слоями (`current_level`). Перед началом обработки слоя его вершины сортируются по возрастанию степеней для минимизации ширины полосы матрицы.
- при просмотре соседей текущей вершины отбираются только локальные узлы текущего процесса. Они сортируются по возрастанию степеней, после чего последовательно добавляются в очередь следующего уровня и фиксируются в векторе `new_order`.
- после обхода всех локальных вершин вектор `new_order` инвертируется (`std::reverse`), завершая формирование порядка Reverse Cuthill-McKee для оптимального сжатия профиля матрицы.

После формирования вектора перестановок выполняется переименование вершин графа для построения новой глобальной нумерации. Каждый процесс сопоставляет свои локальные вершины исходным глобальным ID и обменивается данными через `MPI_Allgatherv`. На основе полученного массива строится `old_to_new_id`, который используется для замены идентификаторов смежных вершин в `endV` на новые значения.

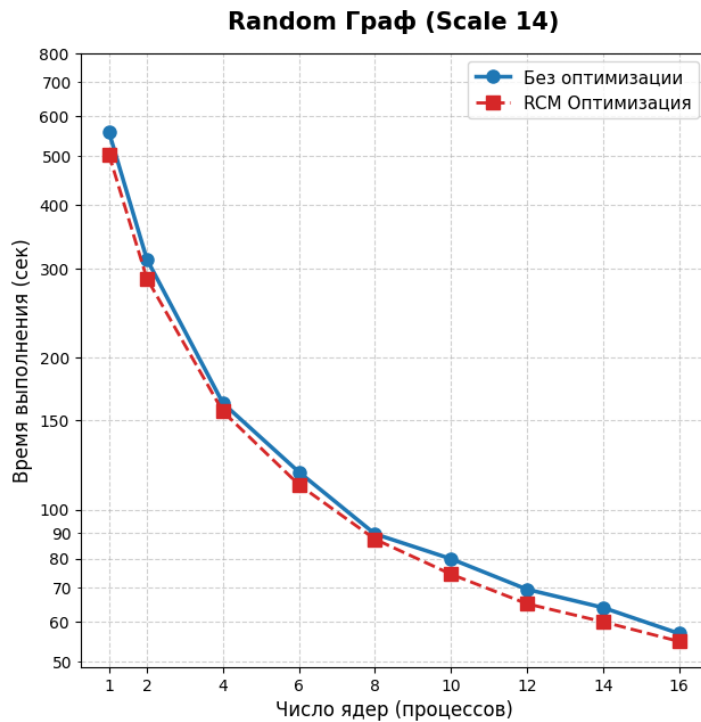
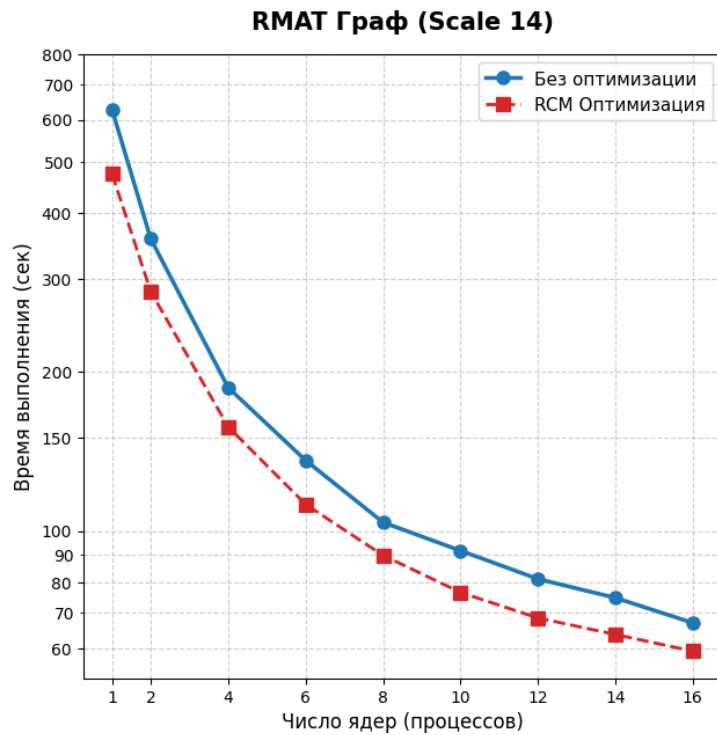
На основе `new_order` создаются новые массивы, куда последовательно переносятся блоки рёбер для улучшения локальности данных. Параллельно формируется `mapping_to_return`, фиксирующий новое положение исходных локальных вершин и используемый для восстановления порядка результатов. В конце списки смежности в `endV` сортируются, что повышает кэш-эффективность и снижает число промахов памяти в основном алгоритме.

6.3 Проверка корректности программы с оптимизацией

```
via@LAPTOP-5613KMS7:/mnt/c/Programs/C/VS_Code/2026-PLGP-BC/2023-PLGP-BC$ mpirun -n 4 ./solution_mpi -in rmat-12
start algorithm...
iter number = 0 finished. Time is 8.44376 sec.
algorithm iterations finished.
rmat-12 vertices = 4096, edges = 131072, nIters = 1, time: min = 8.44375, avg = 8.44375, max = 8.44376
Time = 8.44375 sec.
via@LAPTOP-5613KMS7:/mnt/c/Programs/C/VS_Code/2026-PLGP-BC/2023-PLGP-BC$ ./validation -ans rmat-12.ans -res rmat-12.res
Accepted
```

6.4 Результаты измерения производительности

При масштабировании от 2 до 16 ядер использовались два вычислительных узла, на каждом из которых задействовалось одинаковое количество ядер: от 1 до 8 соответственно. Запуск на 1 ядре выполнялся на одном узле.



6.5 Сравнение производительности

На основании проведенных экспериментов и анализа влияния RCM-оптимизации можно сделать следующие выводы:

Результат применения RCM-оптимизации

Оптимизация на основе алгоритма RCM улучшает локальность доступа к памяти за счёт более компактного расположения связанных вершин. Благодаря этому повышается эффективность использования кэш-памяти процессора и уменьшается количество обращений к оперативной памяти. Эффект от оптимизации наиболее заметен при обработке больших объёмов данных.

Влияние на графы R-MAT

Для графов типа R-MAT оптимизированная версия показывает прирост производительности на всех запусках. Наибольший выигрыш наблюдается при использовании малого числа ядер: время выполнения уменьшается примерно на 24% на одном ядре и на 20% на двух ядрах. При дальнейшем увеличении числа ядер ускорение сохраняется, однако его величина постепенно уменьшается, что объясняется ростом накладных расходов параллельного выполнения, а также сокращением объёма данных, обрабатываемых каждым ядром.

Влияние на случайные графы (Random)

Структура случайных графов хуже подходит для данной оптимизации, поскольку RCM-оптимизация не приводит к существенному уменьшению ширины ленты матрицы смежности. Тем не менее даже в этом случае удаётся добиться прироста производительности на всех конфигурациях, хотя величина ускорения оказывается меньше, чем для графов типа R-MAT и достигает 10% на 1 ядре и 8% на 2 ядрах.